SentinelX SIEM

Platform Presentation Script

Security Information and Event Management Correlation Engine - Project code CIS-117.

A page-by-page walkthrough of the platform: what every page does, how it works end to end, and a ready-to-use talk track for a live demonstration.

Audience: faculty evaluator, technical interviewer, or a developer seeing the system for the first time.

Key framing: detection is DETERMINISTIC rule logic - there is NO machine learning and no AI verdict. Every alert is explainable and linked to the exact events that triggered it. All demo data is local and clearly flagged as synthetic.

Readiness: ready for a CONTROLLED, access-restricted demo. The backend has no authentication yet, so it must not be exposed publicly. The app is configured for Vercel + Railway but has NOT been deployed.

Document generated: 2026-10-18.

1. What SentinelX is (30-second opener)

"SentinelX is a deterministic SIEM correlation engine. It collects security events, normalizes them into one canonical schema, stores them, and correlates them with five explicit rules to produce explainable, evidence-linked alerts. An analyst can then investigate each incident, change its status, and export the evidence. It is rule-based and reproducible - no machine learning - so every alert can be justified exactly."

The three ideas to land

- Correlation: single log lines are noise; the value is connecting many events into one incident.
- Explainability: each alert stores the reason, threshold, window, and the exact evidence events.
- Honesty: synthetic demo data is always flagged; failures are shown as failures, never as fake success.

2. How it works (the big picture)

Architecture

- Frontend: React 19 + Vite + TypeScript + Tailwind (SPA). Presentation only; talks to the backend over HTTP under /api.
- Backend: Python 3.10.7 + FastAPI + SQLAlchemy. Layered: thin API routes -> normalization -> pure detection engine (5 rules) -> repositories.
- Storage: a single SQLite database file. Repositories own all database access; detection rules are pure and never touch the DB.

The pipeline (end to end)

External event or synthetic scenario -> validate + normalize -> persist -> EXPLICIT rule evaluation -> correlated alert created/deduplicated -> evidence linked -> investigation + status update -> analytics / JSON export.

- Nothing auto-correlates: ingestion only stores events. Correlation runs only when a user calls POST /api/alerts/evaluate.
- Deterministic: same stored events + same rule config always produce the same alerts and the same fingerprints.
- Deduplication: each alert has a stable fingerprint (rule id + correlation entities); re-evaluating unchanged evidence is idempotent and preserves analyst status.
- Provenance: every event carries is_synthetic; the API reports synthetic vs real counts so demo data never masquerades as real telemetry.

3. Application map and navigation

SentinelX has two experiences behind one SPA:

- Public Landing page (route /) - a marketing/explainer page for first-time viewers.
- SOC Dashboard (route /dashboard) - the operational console with six sections.

Dashboard shell

Left sidebar (brand + navigation)

- Brand: SENTINELX / SIEM, tagline 'SOC Command Center'. Footer shows 'Engine v0.1.0', 'CIS-117', 'Deterministic Correlation'.
- Six nav items: SOC Overview, Alerts & Incidents (badge = alert count), Event Explorer (badge = event count), Detection Rules, Log Ingestion, Simulation Lab (badge = 5). A link returns to the public landing page.

Top bar

- Shows the current section's title and subtitle, a live UTC clock, a purple 'DEMO ENVIRONMENT' tag, a health pill (SYNCING / API ONLINE / API OFFLINE driven by GET /api/health), and a manual Refresh button.
- If the backend is unreachable, an offline banner appears with a Retry action.

4. Page-by-page walkthrough

4.1 Landing Page (public)

Purpose

Introduce the product to a first-time viewer and route them into the console. It is an explainer, not an operational tool.

What is on the page

- Hero: headline 'See the Threat. Connect the Evidence.' with primary button 'Open Security Console' and a secondary 'Explore Detection Capabilities'.
- Live telemetry ribbon: four counters - Normalized Telemetry (total events), Active Incidents (total alerts), Critical Severity, and Deterministic Rules (5) - animated from the REAL backend stats when the API is online.
- Decorative widgets: a 'LIVE TELEMETRY' ticker and a radar sweep - both explicitly labelled SAMPLE / ILLUSTRATIVE (not real data).
- Detection Capabilities: an interactive selector for the five rules with condition, correlated entities, kill-chain phase, and MITRE ATT&CK mapping.
- Threat Scenarios carousel (forensic SAMPLE case studies), a 6-stage 'How Correlation Works' diagram, an evidence-model preview, and an architecture section.

How it works (data and APIs)

- The counter ribbon reads from the shared app state populated by GET /api/alerts/stats and GET /api/events/stats; if offline it shows a clear 'backend offline' notice instead of fake numbers.
- The sample ticker/radar are static client-side visuals, clearly labelled, to avoid implying a live feed.
- 'Open Security Console' switches the SPA route to /dashboard.

What to say

"This is the public landing page. These four counters are real values from the backend. Notice the ticker and radar are explicitly labelled SAMPLE - we never present illustrative visuals as live data. The five rules here map to MITRE ATT&CK techniques."

What to show

Point to the SAMPLE labels, hover the rule selector, then click 'Open Security Console'.

4.2 SOC Overview (dashboard home)

Purpose

A single-screen situational picture: how many events and alerts exist, their severity and status mix, and recent activity.

What is on the page

- Six KPI cards: Total Telemetry, Recent Activity (last 24h), Total Alerts, Critical Severity, High Severity, Open Lifecycle.
- Alerts by Investigation Status - proportional bars per status.
- Events by Provenance - synthetic vs real split.
- Event Volume Over Time (hourly area chart), Alerts by Severity (donut), Incidents by Detection Rule (bar chart).
- Two feeds: Recent Correlated Alerts and Recent Security Telemetry (click to deep-link into Alerts or Event Explorer).

How it works (data and APIs)

- All numbers come from live API calls: GET /api/alerts/stats (severity/status/rule breakdowns) and GET /api/events/stats (totals, recent, synthetic/real).
- Charts are computed from the fetched stats and recent lists - there are no hardcoded values; empty states render when there is no data.
- Clicking a feed row sets a deep-link target and switches to the relevant section.

What to say

"The Overview is driven entirely by live stats endpoints. The Events-by-Provenance panel is important for honesty - it shows how much of the data is synthetic versus real. Nothing here is hardcoded; with an empty database these panels show empty states."

What to show

Point out the provenance split and the status bars; click a recent alert to jump into Alerts.

4.3 Alerts & Incidents

Purpose

The analyst workspace: review correlated alerts, inspect their evidence, and drive the investigation lifecycle.

What is on the page

- Filter bar (severity, status, rule) and a 'RUN CORRELATION' button that opens a confirm/run modal.
- Alerts table: ID, Severity, Rule, Entities, Events (matched count), Detected At, Status, Provenance, and a Details action.
- Detail side panel: investigation status switcher, detection explanation (reason/threshold/window), entities (source IP, account, first/last seen), an 'Export Alert + Evidence (JSON)' button, and a chronological Evidence Timeline.
- Each evidence row shows the event, a provenance badge, a 'Raw JSON' inspector, and 'Open in Events ->' cross-navigation.

How it works (data and APIs)

- List: GET /api/alerts with pagination and filters. Detail: GET /api/alerts/{id} returns the alert plus its resolved evidence events.
- Status change: PATCH /api/alerts/{id}; the UI shows the backend-confirmed status (not optimistic) and an inline error on failure. The value persists across a page refresh.
- Export: builds a JSON file (alert + evidence) entirely in the browser - no server call, no third party.
- Timeline is sorted by each event's actual timestamp (ascending), so out-of-order ingestion still reads chronologically.
- 'RUN CORRELATION' calls POST /api/alerts/evaluate - the same explicit evaluation used everywhere.

What to say

"This is where an analyst works. Each alert explains itself - here is the reason, the threshold, and the window - and it carries the exact evidence events in time order. I can change the status; that is a real PATCH to the backend and it survives a refresh. I can also export the alert and its evidence as JSON for a report."

What to show

Open an alert, read the explanation, scroll the evidence timeline, change status then refresh, then export JSON.

4.4 Event Explorer

Purpose

Browse and forensically inspect the normalized event stream that feeds correlation.

What is on the page

- Filters: event type, severity, and a Source IP search box.
- Events table: Event ID (+ uid), Timestamp (UTC), Event Type, Severity, Source IP, Destination, Account/Host, Source Feed, Provenance, and an Inspect action.
- Inspect modal: normalized attribute grid (timestamp, ingested-at, IPs, port, user, host, uid, primary key), a 'Referenced by Alerts' reverse lookup, and the original raw JSON payload with Copy.

How it works (data and APIs)

- List: GET /api/events (pagination + filters, newest first). Single event: GET /api/events/{id}.
- Reverse lookup: scans the most recent alerts (GET /api/alerts?limit=500) and filters by evidence_event_ids; it honestly shows 'scanned N of M most recent' when more alerts exist than were scanned.
- Raw payload is rendered as text (JSON.stringify) - never as HTML - so untrusted event fields cannot inject markup.

What to say

"Event Explorer is the forensic view. Every event keeps its original raw payload verbatim. From any event I can see which alerts reference it, and jump straight to that incident - the link between evidence and alert works in both directions."

What to show

Filter by event type, open Inspect, show the raw JSON and the 'Referenced by Alerts' section.

4.5 Detection Rules

Purpose

A transparent, read-only catalogue of the five correlation rules and their live thresholds.

What is on the page

- A banner ('Deterministic Detection Engine Catalog', '5 Active Rules Loaded').
- One card per rule: rule id, category, severity, ENABLED/DISABLED state, description, 'Detection Objective', 'Correlation Logic', and 'Active Parameters & Window'.

How it works (data and APIs)

- Loads from GET /api/alerts/rules, which returns each rule's effective id, name, severity, enabled flag, and params (thresholds/windows).
- Read-only: rules and thresholds are governed by the backend; the page cannot change them.

What to say

"These are the five rules, loaded live from the backend with their real thresholds. This is the heart of the 'deterministic' claim - each rule is an explicit condition over a time window, not a model. The parameters you see are exactly what the engine uses."

What to show

Scroll the five cards; point to AUTH-001's parameters (threshold 10, window 60s).

4.6 Log Ingestion

Purpose

Manually feed events into the system - a single JSON object, a JSON array, or a JSONL file.

What is on the page

- Two tabs: 'JSON Event / Array Editor' and 'JSONL File Stream Upload'.
- Pre-configured samples, including a deliberately MALFORMED payload for demonstrating validation.
- A result panel: HTTP status (201 CREATED / 207 MULTI-STATUS / FAILED) and counters - Received, Accepted, Duplicates, Rejected - with per-record rejection reasons and the persisted record IDs.

How it works (data and APIs)

- Both tabs call POST /api/events/ingest. Each record is validated and normalized independently; one bad record does not fail the batch.
- Bounds: up to 1,000 events / ~5 MB per JSON batch; up to 10,000 records / ~10 MB per JSONL upload.
- Accepted events are persisted and de-duplicated by event_uid; the response drives the counters.

What to say

"This is how events enter the system in the real world - you would POST from your log shipper. Watch what happens with the malformed sample: the backend rejects just that record with a precise field error and accepts the rest. Ingestion only stores events - it never creates alerts."

What to show

Submit the valid sample (201, Accepted), then the malformed sample (rejected with field errors).

4.7 Simulation Lab

Purpose

A teaching tool that demonstrates the full detection pipeline safely with synthetic attacks.

What is on the page

- An amber 'SIMULATION MODE' banner (synthetic events only, RFC 5737 test IPs).
- Five scenario cards (one per rule). Each run is a three-step stepper.
- A verdict panel that advances through GENERATED -> INGESTED -> EVALUATED -> DETECTED / NOT DETECTED / FAILED, with an 'Open Incident Details' link on a detection.

How it works (data and APIs)

- Step 1 'Generate Scenario' is client-side only (builds synthetic events; no API call, no alert).
- Step 2 'Ingest Events to API' calls POST /api/events/ingest and shows the real counts.
- Step 3 'RUN CORRELATION' calls POST /api/alerts/evaluate then GET /api/alerts - the same engine as the rest of the app.
- The Lab never creates alerts directly, and correlation is always an explicit click. A backend/network error shows a distinct 'Correlation Failed' state - never a false 'no detection'.

What to say

"The Simulation Lab is the cleanest way to show the whole pipeline. I generate a synthetic brute-force scenario, ingest it, and then explicitly run correlation. The verdict comes from the real backend engine. If the backend were down, you would see a FAILED state - we never disguise a failure as a clean 'no detection'."

What to show

Run AUTH-001 end to end: Generate -> Ingest -> RUN CORRELATION -> DETECTED -> Open Incident Details.

5. The five detection rules (reference)

Thresholds are the rules' defaults and can be overridden per evaluation call (enable flags and positive-integer thresholds only). Severity is a property of the RULE, not of the triggering events.

Rule	Severity	Threshold / window	Correlates by
AUTH-001 Brute Force Authentication	high	>= 10 auth failures / 60s	source_ip
AUTH-002 Password Spraying	high	>= 5 distinct accounts / 300s	source_ip (+ username)
AUTH-003 Failed Logins then Success	critical	>= 5 failures then a success / 300s	source_ip + username
NET-001 Potential Port Scan	medium	>= 15 distinct dest ports / 60s	source_ip (+ dest port)
PRIV-001 Suspicious Privilege Escalation	high	priv-esc then sensitive action / 300s	username

- Each alert stores an explanation object: reason, threshold, window, counts, and entities.
- AUTH-003 and PRIV-001 emit at most one detection per correlation key per run.
- PRIV-001 correlates by username, so its alerts have no source IP.

6. Suggested live demo sequence (7-10 minutes)

Tip: for a clean, repeatable demo run the backend against a disposable session database so the persistent demo database is never modified (see docs/demo-runbook.md).

- 1. Start backend + frontend; confirm GET /api/health returns status ok / database connected.
- 2. Landing page: point out the real counters and the SAMPLE labels; click 'Open Security Console'.
- 3. Detection Rules: show the five live rules and thresholds (the 'deterministic' proof).
- 4. Simulation Lab: Generate an AUTH-001 scenario -> Ingest Events to API -> RUN CORRELATION -> DETECTED.
- 5. Open Incident Details: read the explanation and the evidence timeline.
- 6. Change the alert status, refresh the browser, show it persisted; then Export Alert + Evidence (JSON).
- 7. Overview: show Alerts-by-Status and Events-by-Provenance updating from live data.
- 8. Close with the honest limitations (below).

7. Anticipated questions and accurate answers

Q: What makes this a SIEM?

A: It performs the core SIEM loop: centralized ingestion and normalization, persistence, rule-based correlation across many events, and explainable, evidence-linked alerting with an investigation workflow and analytics. It is a focused educational implementation, not an enterprise product.

Q: How are detection rules implemented?

A: As pure Python functions (one per rule) that evaluate events over a sliding time window with explicit thresholds; the engine runs the enabled rules and the rules never touch the database.

Q: How does it avoid duplicate alerts?

A: Each detection has a stable fingerprint (rule id + correlation entities). Evaluation upserts by fingerprint: new -> created, changed evidence -> updated (status preserved), unchanged -> no write. So re-evaluation is idempotent.

Q: Can it ingest real telemetry?

A: Yes - the ingestion API accepts any JSON/JSONL events matching the schema, and every event is flagged synthetic or real. There are no built-in production log-source connectors yet; you POST from your own shipper.

Q: Does it use machine learning?

A: No. Detection is entirely deterministic rules - a deliberate choice for explainability and reproducibility.

Q: Why is correlation triggered manually?

A: To keep the model transparent and idempotent: ingestion only stores events; correlation is a distinct explicit action (POST /api/alerts/evaluate). A scheduler could call the same endpoint.

Q: What happens if the database fails?

A: GET /api/health reports it honestly - HTTP 503 with database 'unavailable' instead of a misleading 200. Writes roll back on error, and in production the app refuses to start on an unsafe ephemeral database path.

Q: Is it production-ready?

A: Not for unrestricted public use. It is a well-tested controlled demo. The main gap is the absence of authentication; there is also no retention or automated backup. These are documented with a plan.

Q: How is authentication handled?

A: It is not implemented yet - the primary known limitation. CORS limits browser origins but is NOT authentication. Keep the backend access-restricted until a real auth layer is added.

Q: How would it scale beyond SQLite?

A: SQLite is single-writer, so today you run one backend instance. Data access is isolated in the repository layer and the database URL is configurable, so moving to PostgreSQL is the natural path; it is not wired up or tested yet, so I would not claim it works today.

8. Honest limitations and future work

Limitations (state these plainly)

- No authentication/authorization - not safe for public exposure; controlled demo only.
- Deterministic rules only - no ML, no anomaly detection.
- SQLite single-writer; no storage retention/quota; no automated backup.
- Deployment is configured for Vercel + Railway but has NOT been deployed or verified.
- The browser acceptance suite was executed in an earlier phase; it was not re-run for this script.

Future work

- Authentication/authorization (or platform access control) before any public use.
- Storage retention + quotas and automated database backup.
- A frontend Content-Security-Policy once the backend origin is known.
- Optional real log-source connectors; PostgreSQL for horizontal scale; CSV/PDF export; scheduled correlation.

9. Quick reference

Start the application (Windows PowerShell)

```
cd backend
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
cd frontend
npm run dev
```

API endpoints (all under /api)

- GET /api/health - liveness + DB connectivity (200 / 503).
- POST /api/events/ingest - ingest event(s) or JSONL (state-changing).
- GET /api/events, GET /api/events/{id}, GET /api/events/stats - read events.
- POST /api/alerts/evaluate - run correlation (state-changing).
- GET /api/alerts, GET /api/alerts/{id}, GET /api/alerts/stats, GET /api/alerts/rules - read alerts.
- PATCH /api/alerts/{id} - update investigation status (state-changing).

Security note: the API has no authentication. CORS restricts browser origins only; it is not authentication and does not stop direct clients. Do not expose the backend publicly without an access-control layer.